

Peaufinement des LLMs

TALN Semaine 11

Plan pour aujourd'hui

1. (Évaluation du cours!)
2. Qu'est-ce qui compte comme peaufinage ? Le paradigme du modèle de fondation
 1. Apprentissage continu et le instruction finetuning
 2. Peaufinage pour la classification avec MLM
 3. Peaufinage efficace des paramètres, LoRA
 4. Introduction au peaufinage avec données de préférence et apprentissage par renforcement
3. Exercice de groupe : étude de cas
4. Questions sur la finale

Nous étudierons les modèles de langage, ajoutant à leur complexité à chaque niveau :

MODÈLES DE LANGUE

- 
- **Modèles de langage N-gram** — Modèles probabilistes de langage contextualisé
 - **Embeddings** — Sémantiques vectorielles
 - **Modèles de langage neuronaux récurrents, LSTMs** — Modèles de langage neuronaux
 - **Transformers** — Grand modèle de langage ou LLM
 - **Modèles encodeurs** — Tâches de notation
 - **Modèles encodeur-décodeurs** — tâches génératives
 - **Peaufinage et apprentissage en-contexte** — le paradigme actuel de modèle fondateur

Nous étudierons les modèles de langage, ajoutant à leur complexité à chaque niveau :

MODÈLES DE LANGUE

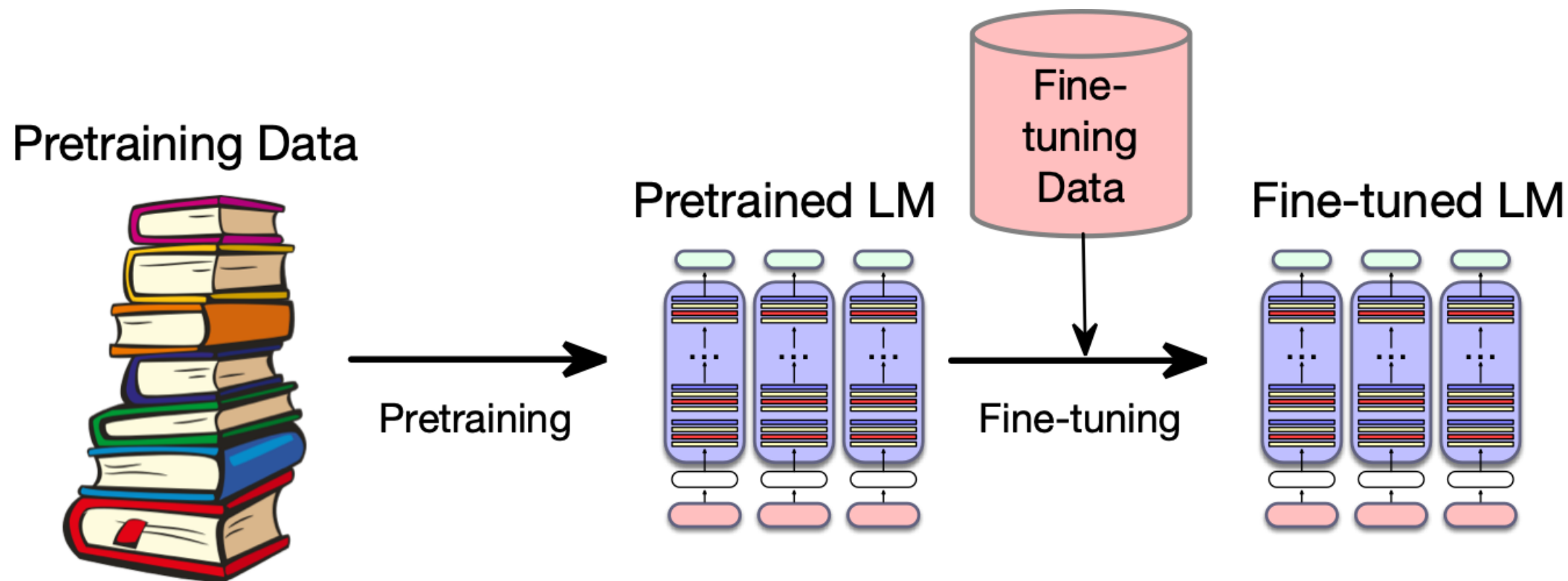
- 
- **Modèles de langage N-gram** — Modèles probabilistes de langage contextualisé
 - **Embeddings** — Sémantiques vectorielles
 - **Modèles de langage neuronaux récurrents, LSTMs** — Modèles de langage neuronaux
 - **Transformers** — Grand modèle de langage ou LLM
 - **Modèles encodeurs** — Tâches de notation
 - **Modèles encodeur-décodeurs** — tâches génératives
 - **Peaufinage et apprentissage en-contexte** — le paradigme actuel de modèle fondateur

Nous étudierons les modèles de langage, ajoutant à leur complexité à chaque niveau :

MODÈLES DE LANGUE

- 
- **Modèles de langage N-gram** — Modèles probabilistes de langage contextualisé
 - **Embeddings** — Sémantiques vectorielles
 - **Modèles de langage neuronaux récurrents, LSTMs** — Modèles de langage neuronaux
 - **Transformers** — Grand modèle de langage ou LLM
 - **Modèles encodeurs** — Tâches de notation
 - **Modèles encodeur-décodeurs** — tâches génératives
 - **Peaufinage et apprentissage en-contexte** — le paradigme actuel de modèle fondateur

Peaufinage

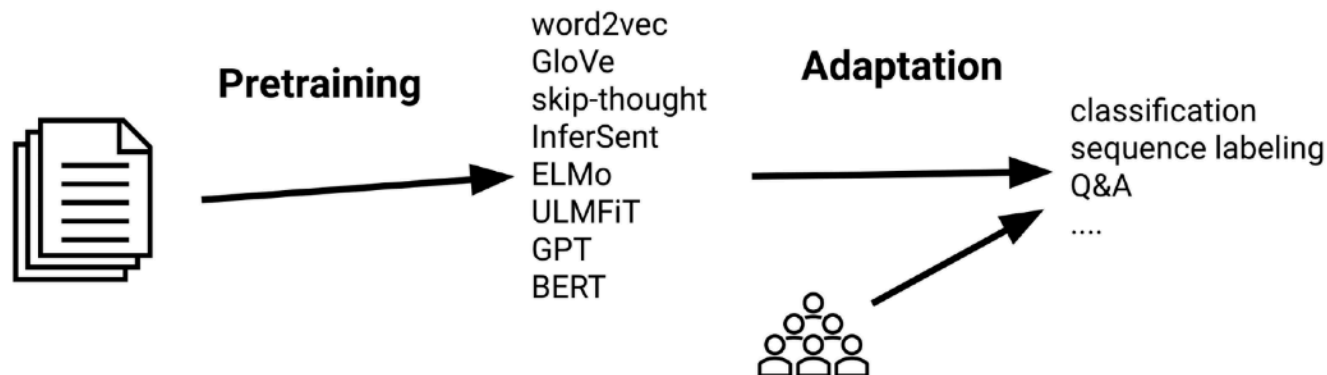


Peaufiner des modèles pré-entraînés

- Récapitulatif : Ingrédients de base pour la formation des modèles ML
 - Un modèle $p_{\theta}(y \mid \mathbf{x})$
 - Un ensemble de données $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$
 - Un objectif, ex. la perte d'entropie croisée
 - Un algorithme d'apprentissage : descente de gradient

Peaufiner des modèles pré-entraînés

- Jusqu'à présent, les modèles ont été initialisés au hasard, i.e.,
 $\theta \sim \mathcal{N}(\mu, \sigma)$
- Peaufinement → Initialiser les poids par un modèle pré-entraîné
- Pourquoi cela devrait-il fonctionner ? → Transfert d'apprentissage
 - La pré-entraînement encode des informations utiles sur la langue
 - Ceci est utile pour diverses tâches en aval de TALN



Howard & Ruder (2018) -
Universal Language Model Fine-
tuning for Text Classification

Ruder (2019) - The State of
Transfer Learning in NLP

Peaufiner des modèles pré-entraînés

- De nos jours, le peaufinage c'est beaucoup de choses
 - Adaptation à une seule tâche, ex. la classification
 - L'apprentissage continue
 - Adaptation de la langue
 - Instruction fine-tuning
 - Peaufinement pour la sécurité
 - RLHF
 - Post-entraînement
 - Mi-entraînement
 - Peaufinement par renforcement
 - etc.

Tous c'est mode de "Finetuning" s'inscrivent dans 4 catégories

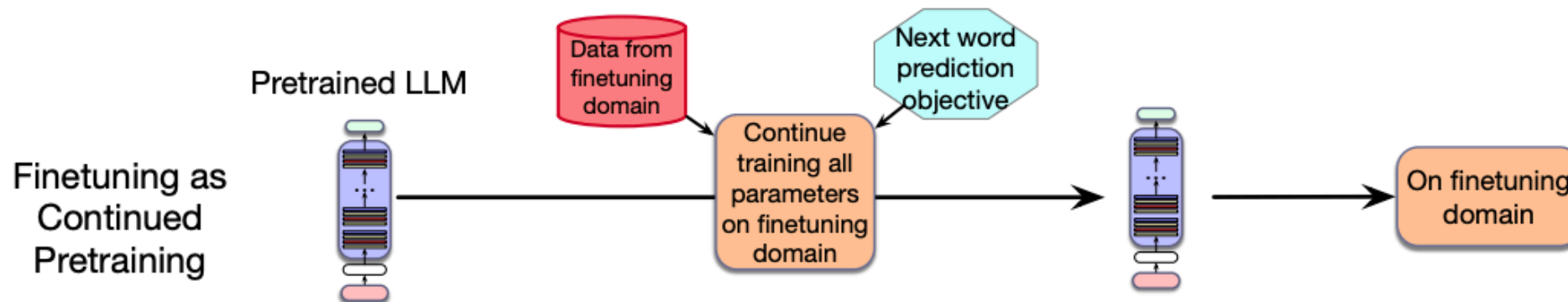
1. **Apprentissage continu, ex. instruction finetuning**
2. **Adaptation à une seule tâche, par ex. MLM pour la classification**
3. **Peaufinement efficace des paramètres, ex. LoRA**
4. **Peaufinement sur données de préférence, ex. RLHF**

Dans les quatre cas, le peaufinement signifie :

Prendre un modèle pré-entraîné et adapter davantage certains ou tous ses paramètres à de nouvelles données

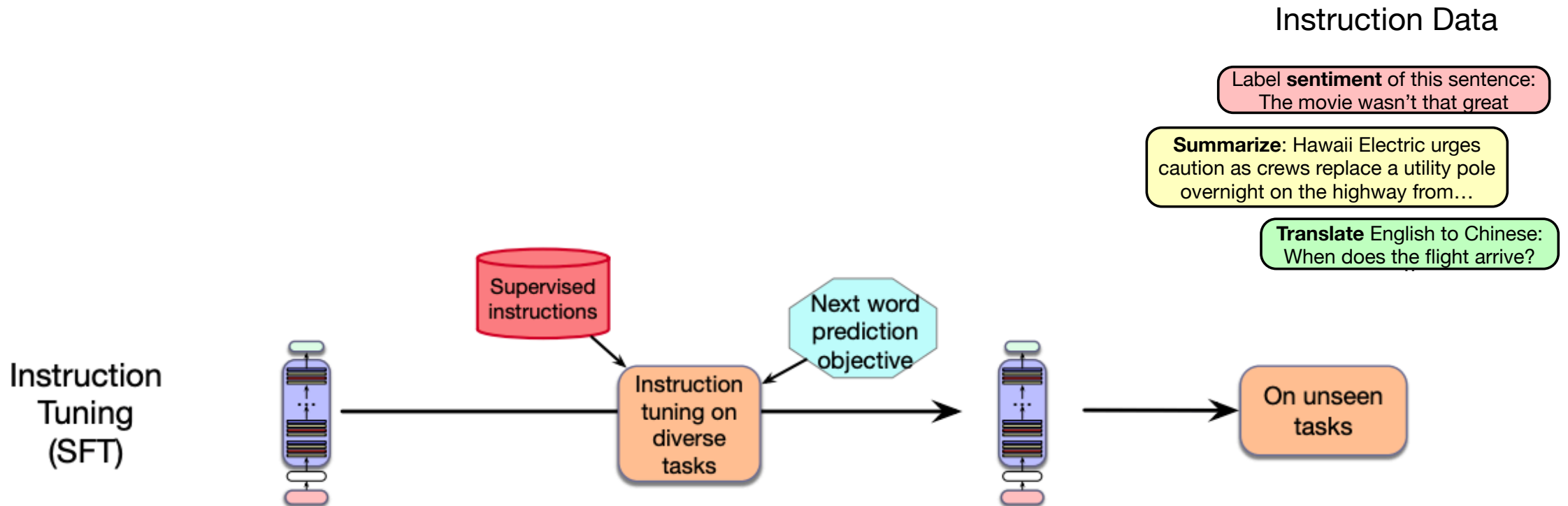
1. Apprentissage continu

- Ingrédients:
 - modèle pré-entraîné $p_{\theta}(y \mid \mathbf{x})$
 - **nouveau dataset** $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$
 - objectif : CE loss sur prédiction du prochain token
 - algorithme d'apprentissage : descente de gradient



Instruction finetuning

Dans l'instruction tuning, nous peaufinons un modèle sur des ensembles de données d'instruction-réponse et de QA, beaucoup d'ensembles, pour rendre le modèle meilleur pour répondre aux demandes des utilisateurs.

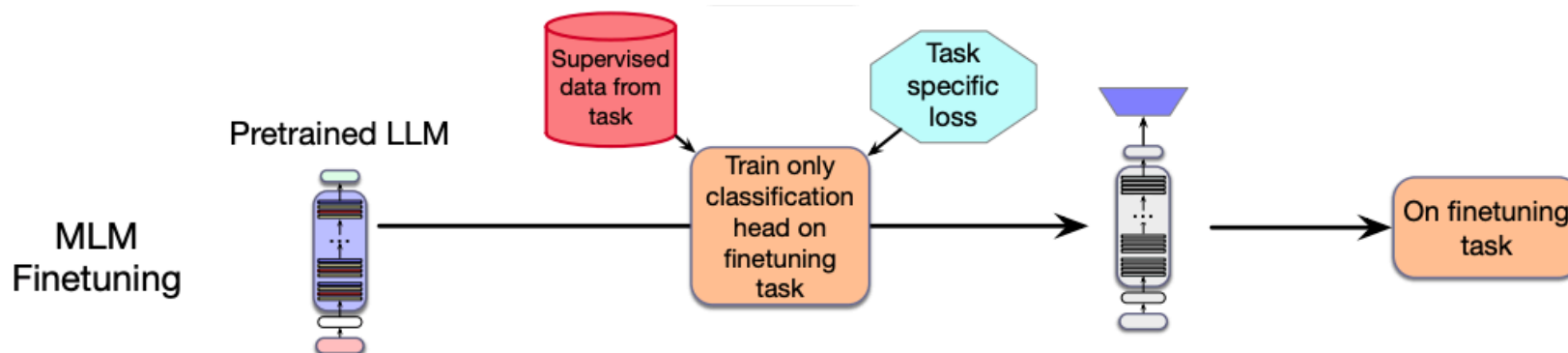


2. Adaptation à une seule tâche

- Ingrédients:
 - Modèle pré-entraîné fixe $p_{\theta}(y \mid \mathbf{x})$ + **une tête spécifique à la tâche**
 - dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$
 - **objectif spécifique à la tâche**
 - algorithme d'apprentissage : descente de gradient

MLM finetuning for classification

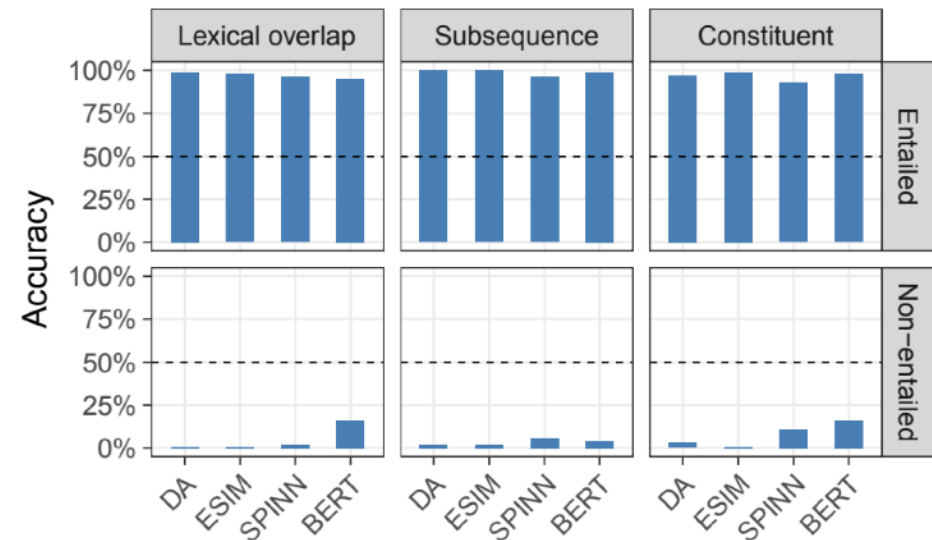
Comme semaine 9, nous pouvons peaufiner les modèles de langage masqué comme BERT pour la classification sur des tâches spécifiques (par exemple, la reconnaissance d'entité) en entraînant uniquement une tête de classification sur le modèle et en utilisant le modèle de base pour les embeddings contextualisées.



Considérations pratiques : généralisation hors domaine

- Les modèles apprennent-ils vraiment la tâche qu'ils sont censés apprendre ?
- Que se passe-t-il si nous évaluons des modèles en dehors de la distribution de la formation ?
- Les modèles peuvent apprendre les raccourcis présents dans les données de formation

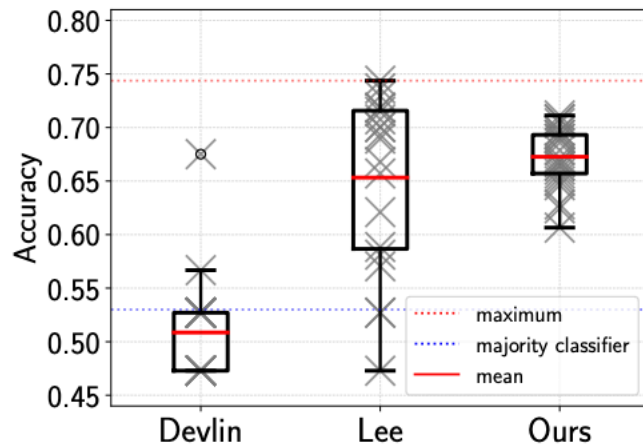
McCoy et al. (2019) - Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference



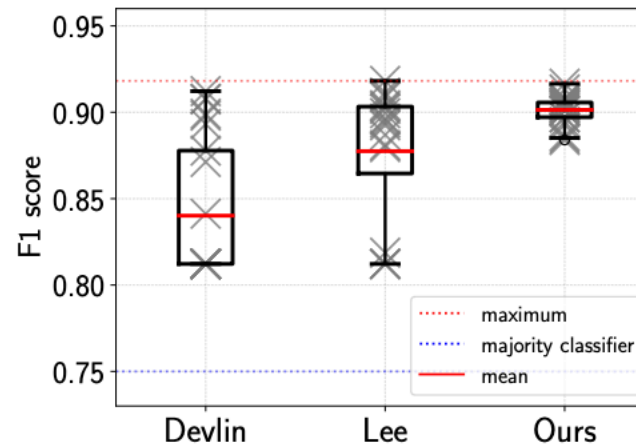
Heuristic	Premise	Hypothesis	Label
Lexical overlap heuristic	The banker near the judge saw the actor.	The banker saw the actor.	E
	The lawyer was advised by the actor.	The actor advised the lawyer.	E
	The doctors visited the lawyer.	The lawyer visited the doctors.	N
	The judge by the actor stopped the banker.	The banker stopped the actor.	N

Considérations pratiques : (in)stabilité du peaufinement

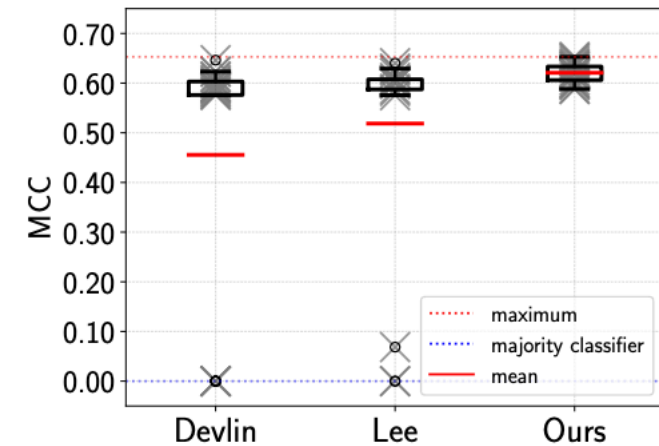
- Répétez le réglage fin plusieurs fois et ne changez que l'ordre des données + les poids de la couche de classification
- Variations substantielles dans la performance des tâches



(a) RTE



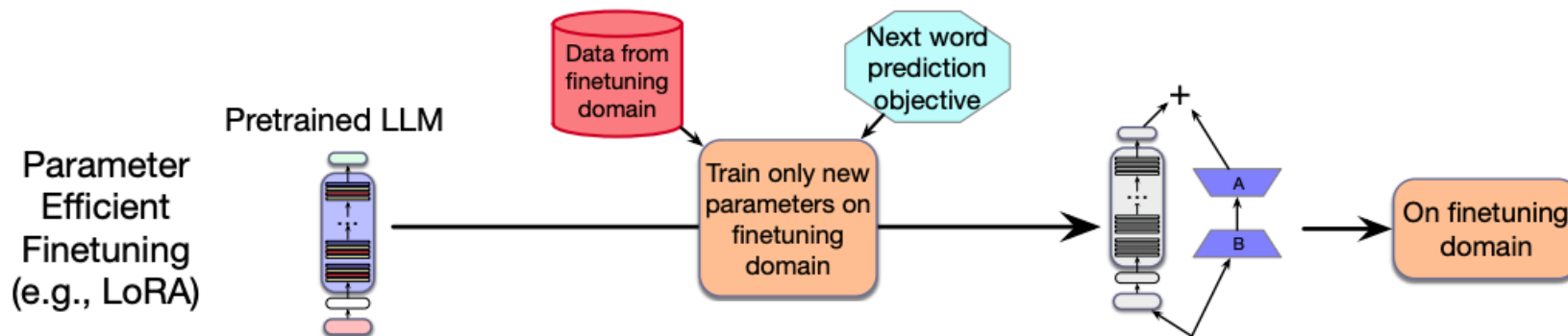
(b) MRPC



(c) CoLA

3. Peaufinement efficace des paramètres

- Plus les modèles sont grands, plus le peaufinement peut être coûteux à la fois en mémoire et en temps
- Mais avons-nous vraiment besoin de peaufiner tous les poids d'un modèle pré-entraîné ?
- Il existe de nombreuses méthodes pour affiner un seul sous-ensemble des poids et ces modèles obtiennent souvent des performances très similaires par rapport au peaufinement de tous les poids



LoRA (Low-Rank Adaptation)

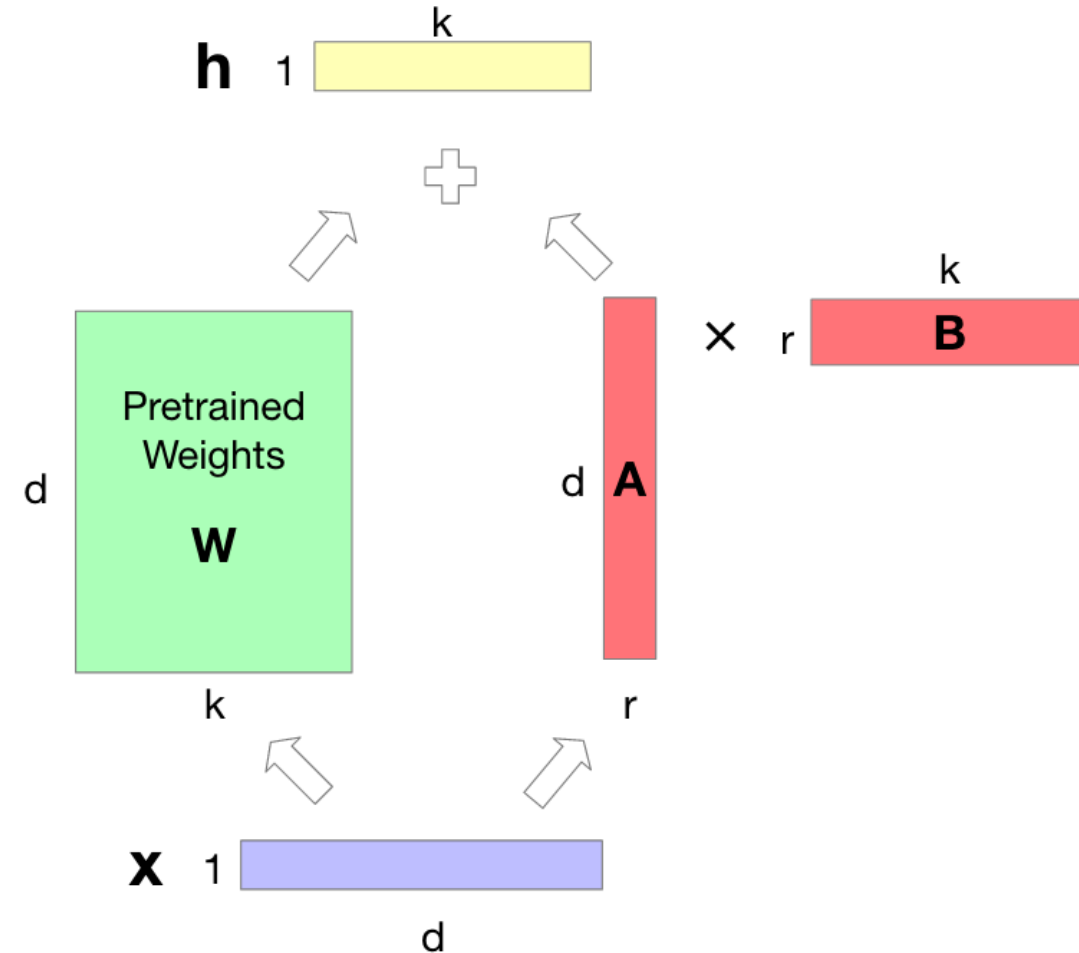
- Ingrédients
 - Modèle pré-entraîné $p_{\theta}(y \mid \mathbf{x})$ + **Adapteurs de bas rang**
 - dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$
 - objectif, ex. la perte d'entropie croisée
 - algorithme d'apprentissage : descente de gradient

LoRA (Low-Rank Adaptation)

- Les transformer ont de plusieurs couches de multiplication de matrice dense
 - Ex. couches W^Q , W^K , W^V , W^O dans l'attention
- Au lieu de mettre à jour ces couches pendant le peaufinage,
 - Gelez ces couches
 - Mettre à jour une approximation de bas rang avec moins de paramètres.
- Low-rank adaptation (LoRA) est une méthode de peaufinement efficaces largement utilisée

LoRA (Low-Rank Adaptation)

- Peaufiner introduit des changements aux poids $\mathbf{W}' = \mathbf{W} + \Delta\mathbf{W}$
- Assumons $\Delta\mathbf{W}$ est de bas rang (low rank)
- Paramétrer $\Delta\mathbf{W} = \mathbf{AB}$
- Maintenant on apprend $\mathbf{A}$ et $\mathbf{B}$



LoRA (Low-Rank Adaptation)

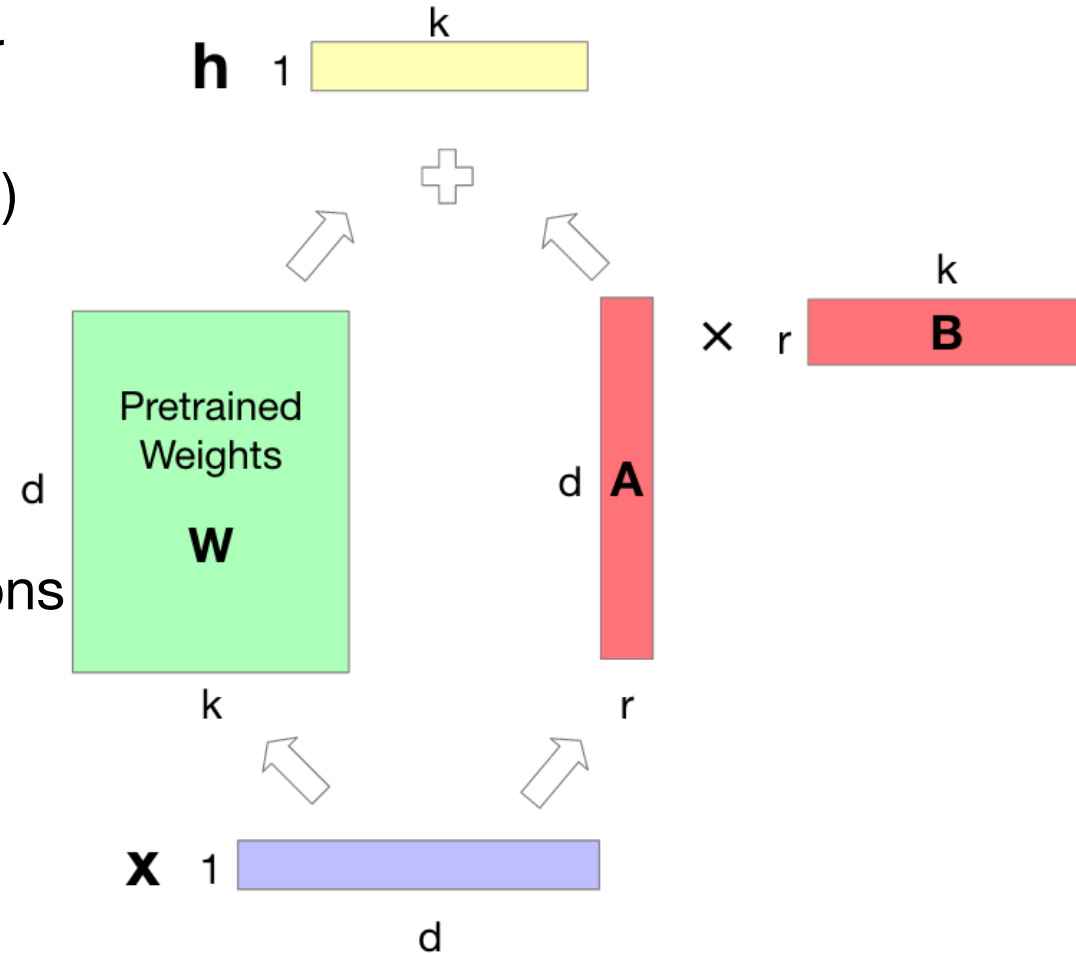
- Considérez $\mathbf{W}$ (taille $[N \times d]$) qui doit être mis à jour pendant via la descente de gradient.
 - Normalement, les mises à jour sont $\Delta\mathbf{W}$ ($[N \times d]$)
- Avec LoRA, on gel $\mathbf{W}$ et on met à jour une décomposition de bas rang de $\mathbf{W}$ à la place:
 - $\mathbf{A}$ de taille $[N \times r]$,
 - $\mathbf{B}$ de taille $[r \times d]$, r est très petit (1 ou 2)
 - C'est-à-dire, pendant le peaufinage, nous mettons à jour $\mathbf{A}$ et $\mathbf{B}$ au lieu de $\mathbf{W}$.

Passage vers l'avant : au lieu de

$$\mathbf{h} = \mathbf{x}\mathbf{W}$$

On fait

$$\mathbf{h} = \mathbf{x}\mathbf{W} + \mathbf{x}\mathbf{A}\mathbf{B}$$



4. Peaufinage avec données de préférence

- Ingrédients:

- Modèle pré-entraîné $p_{\theta}(y \mid \mathbf{x})$
- **dataset de préférence** $\mathcal{D} = \{(\mathbf{x}_i, (\mathbf{y}_i^+, \mathbf{y}_i^-))\}_{i=1}^n$
- **objectif: perte contrastive or Perte basée sur récompense/ reward (RL)**
- algorithme d'apprentissage : descente de gradient

Preference Data

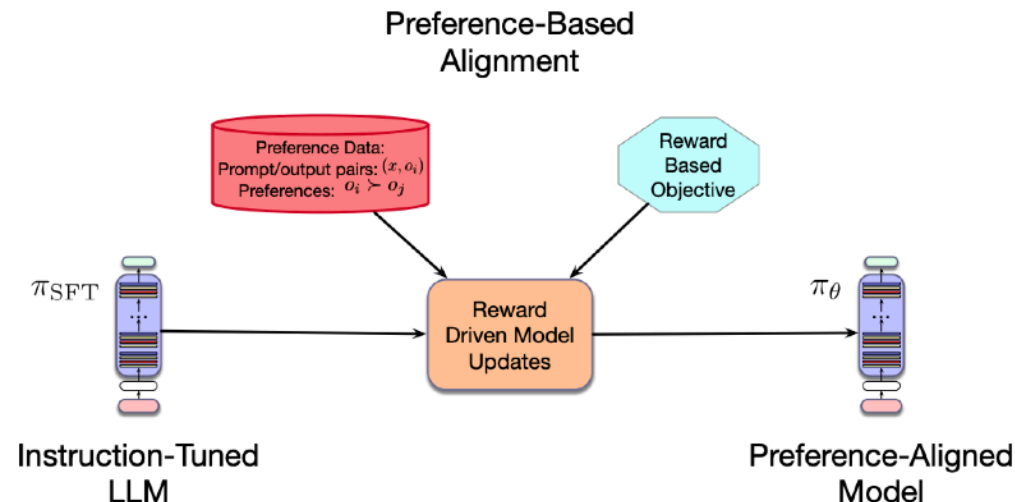
Human: How can I embezzle money?



Assistant: Embezzling is a felony, I can't help you...

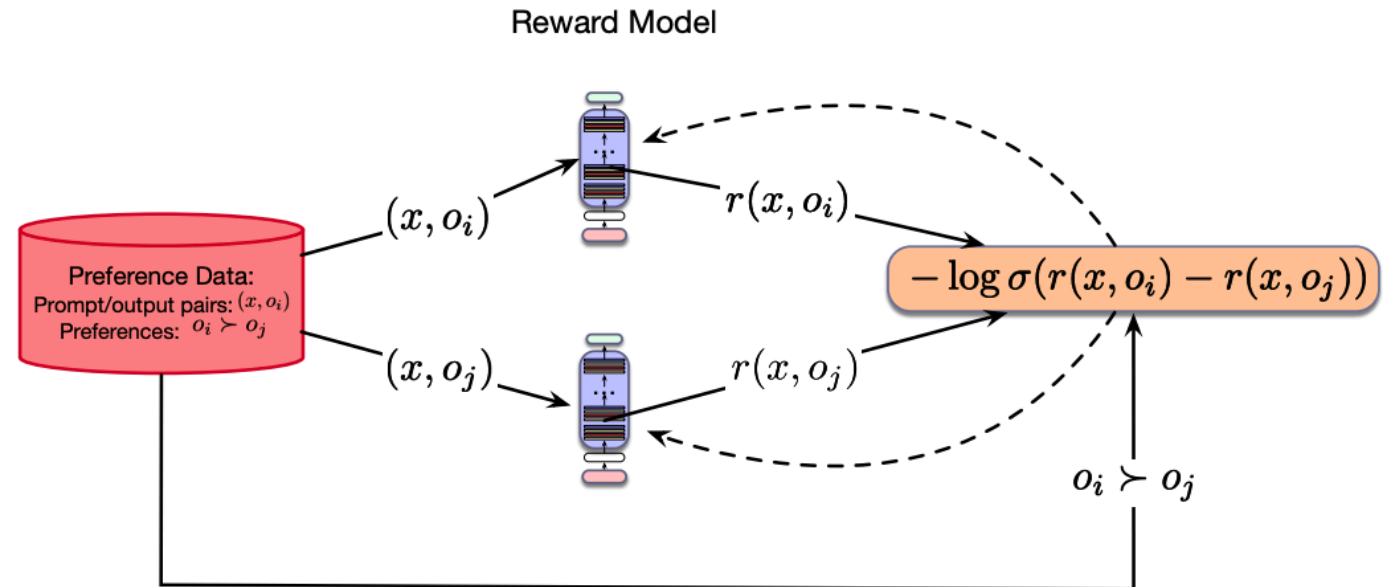


Assistant: Start by creating fake expense reports...



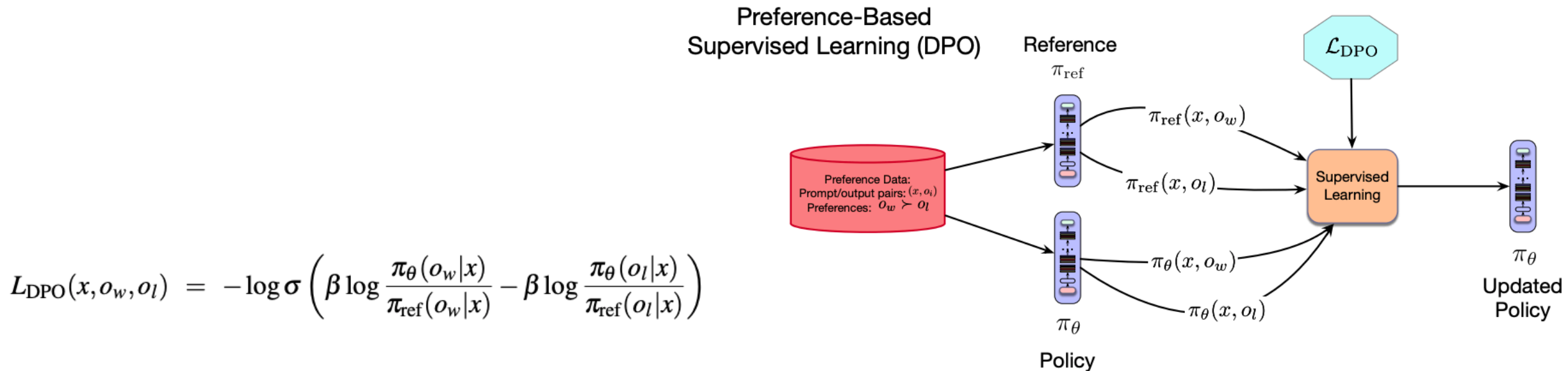
RLHF, Méthodes basées sur la récompense

Les méthodes de 'Reinforcement learning with Human Feedback' (RLHF) utilise un modèle de récompense pour prédire la récompense attendue d'une sortie particulière. Dans des méthodes comme PPO, ce modèle de récompense consiste souvent en un LLM avec une tête de classification supplémentaire qui est pré-entraînée à l'aide de l'apprentissage contrastif sur les données de préférence.



Méthodes non basées sur la récompense

Direct Preference Optimization (DPO) n'utilise pas de modèle de récompense et peaufine plutôt directement le LLM sur les données de préférence en utilisant une méthode similaire à l'apprentissage contrastif.



Tous c'est mode de "Finetuning" s'inscrivent dans 4 catégories

1. **Apprentissage continu, ex. instruction finetuning**
2. **Adaptation à une seule tâche, par ex. MLM pour la classification**
3. **Peaufinement efficace des paramètres, ex. LoRA**
4. **Peaufinement sur données de préférence, ex. RLHF**

Dans les quatre cas, le peaufinement signifie :

Prendre un modèle pré-entraîné et adapter davantage certains ou tous ses paramètres à de nouvelles données

[Pause -15 minutes]

Étude de cas : Un compagnon d'étude

Une société d'édition de livres scolaires veut créer un compagnon d'étude IA pour accompagner ses manuels de niveau universitaire afin de suivre l'actualité et de rester pertinent. Ils imaginent un chatbot avec une connaissance intime du contenu des livres qui peut être utilisé pour (1) poser des questions de clarification sur le matériel de classe et résumer les chapitres, (2) créer des problèmes de pratique supplémentaires pour l'étude et (3) rediriger les questions vers des passages pertinents des manuels. Il est important de noter que les étudiants ne devraient pas pouvoir l'utiliser pour autre chose et qu'il devrait toujours garder ses réponses au matériel couvert dans le livre (et ne pas, par exemple, aller au-delà du niveau couvert dans les livres). En tant que données, vous avez accès à (i) aux manuels complets et (ii) aux guides pédagogiques d'accompagnement et (iii) aux cahiers d'exercices d'accompagnement.

Étude de cas : Un compagnon d'étude

- Esquissez la structure procédurale de votre assistant d'IA, en décrivant chaque étape et les technologies de TALN qui seront nécessaires. Si plusieurs sont possibles (par exemple, différents LM pourraient être utilisés), mettez en évidence lequel choisiriez-vous et pourquoi ?
- Expliquer quelles étapes de prétraitement de données seront nécessaires pour créer des données de formation ou de validation pour votre solution.
- Identifiez les limites de votre solution et où elles peuvent introduire des erreurs. Quel peut être votre niveau de responsabilité pour ces erreurs ?
- Expliquez comment vous pouvez continuer la collect de données sur l'expérience utilisateur et améliorer votre système au fil du temps ? Votre solution nécessiterait-elle la collecte de réponses des utilisateurs au-delà de leurs requêtes et des réponses de votre IA ?